MSc IN ARTIFICIAL INTELLIGENCE · MACHINE LEARNING ASSIGNMENT

Explainable Used-Vehicle Price Prediction in Sri Lanka

A glass-box Explainable Boosting Machine compared with black-box tree ensembles



Figure 1. *Project lifecycle: scraped ikman.lk listings → cleaned dataset → evaluated and explained models → Streamlit front end.*

Report information	Details
Module	MSc in Artificial Intelligence – Machine Learning Assignment
Task type	Supervised tabular regression with explainable AI (XAI)
New algorithm	Explainable Boosting Machine (EBM, InterpretML) – not covered in lectures
Dataset	10,388 used-car advertisements scraped from ikman.lk (8,554 after cleaning)
Deliverables	Two Jupyter notebooks, saved models and evaluation artefacts, live Streamlit web app, demo video
Live application	https://srilankavehiclepricepredictionproject-qpxf6lfrt2547rgriw7pir.streamlit.app/
Source code (GitHub)	https://github.com/VDARW-UOM/Sri_Lanka_Vehicle_Price_Prediction_Project
Prepared	November 2026
Student / Registration no.	Akith Wijesundara · [Add registration number]

EXECUTIVE SUMMARY

Abstract

This project builds an end-to-end, explainable machine learning prototype that estimates the advertised asking price of used cars in Sri Lanka. The dataset was scraped from ikman.lk, a Sri Lankan online classifieds marketplace, and contains 10,388 car advertisements with ten descriptive attributes and an asking price. After auditable cleaning (removal of 1,833 duplicate records and one extreme price-entry error), 8,554 listings remained.

The core of the study is an **Explainable Boosting Machine (EBM)**, a glass-box generalised additive model that was not taught in the module. EBM is trained, tuned and tested under exactly the same leakage-aware protocol as a median baseline, a Decision Tree and three black-box tree ensembles (Random Forest, XGBoost and LightGBM). Data are split into group-disjoint training, validation and test partitions (5,995 / 1,274 / 1,285 listings); the deployed price model is chosen on validation mean absolute error (MAE) only.

The default-settings XGBoost obtained the lowest validation MAE (LKR 1.87 million) and was selected. On the held-out test set it achieved MAE LKR 1.97 million, RMSE LKR 6.68 million and R^2 0.901. EBM was markedly less accurate (test MAE LKR 4.99 million, R^2 0.755) but is fully transparent: every EBM estimate is an exact sum of a baseline and one contribution per feature. XAI analysis (permutation importance for both models, EBM shape functions and exact local explanations) shows that both models rely on the same five drivers: engine capacity, manufacture year, brand, mileage and model. The trained models are integrated into a Streamlit web application, live on Streamlit Community Cloud, that returns a price estimate together with its explanation.

PROJECT AT A GLANCE

Measure	Observed result
Raw → cleaned listings	10,388 → 8,554 (82.35% retained)
Inputs and target	10 predictors (7 categorical, 3 numeric); target PriceLKR (asking price, LKR)
Data split (group-disjoint)	5,995 train / 1,274 validation / 1,285 test
New algorithm	Explainable Boosting Machine – test MAE LKR 4.99M, R^2 0.755
Validation-selected price model	XGBoost (default settings) – test MAE LKR 1.97M, R^2 0.901
XAI methods	Permutation importance (both models), EBM term importance, shape functions, local explanations
Front end	Live Streamlit app on Streamlit Community Cloud: prediction, EBM explanation, model comparison, limitations

REPORT ROADMAP

1 Problem definition • 2 Data collection and preprocessing • 3 Selection of a new algorithm (EBM) • 4 Training and evaluation • 5 Explainability and interpretation • 6 Critical discussion • 7 Front-end integration • 8 Reproducibility • 9 Conclusion

1. Problem Definition and Relevance

1.1 The problem

Sri Lanka has an active used-car market in which most buyers and sellers first meet through online classified advertisements. Prices are hard to judge: they depend on interacting characteristics such as make, model, trim, age, engine capacity and mileage, and they shifted sharply during the vehicle-import restrictions of 2020–2025. A buyer has no easy way to tell whether an asking price is reasonable, and a seller has no easy way to choose one. A single rule based on age or mileage cannot capture all of these combinations.

The study therefore frames price estimation as **supervised regression**: given a record of vehicle attributes *X*, estimate its numeric asking price *y = PriceLKR* in Sri Lankan rupees. Because a price estimate is only useful if people can trust it, the project gives equal weight to **explaining** the prediction: which attributes push the price up or down, and by how much.

1.2 Relevance

- **Buyers and sellers** get an indicative, data-based reference price for a specific vehicle configuration.
- **Dealers and marketplaces** could flag listings whose asking price is far from comparable vehicles, which may indicate data-entry errors or misleading advertisements.
- **Explainability** turns a black-box number into a breakdown that a non-specialist can check against their own knowledge of the market.

1.3 Objectives and scope

1. Collect and clean a local dataset of Sri Lankan car advertisements with an auditable set of rules.
2. Apply a machine learning algorithm not covered in lectures (EBM) and compare it fairly with standard tree-based models.
3. Select the deployed model on validation data only, then report unbiased held-out test performance.
4. Explain what the models learned with global and local XAI methods and check the explanations against domain knowledge.
5. Integrate the trained models into a web front end that returns predictions and explanations.

The target is the **advertised asking price**. The project does not predict negotiated sale prices, insurance or bank valuations.

2. Dataset Collection and Preprocessing

2.1 Data source and how it was collected

The dataset was scraped from **ikman.lk**, a widely used Sri Lankan online classifieds marketplace, from its car advertisements section. An automated scraper visited the public advertisement pages and extracted the structured specification fields shown on each listing (brand, model, edition, year of manufacture, transmission, fuel type, body type, engine capacity, mileage, price and whether the price is negotiable). The records were saved to a single CSV file, ikman-scrapes.csv, which is the raw input to Notebook 01.

The raw file contains 10,388 records and 11 columns. Values are stored as they appear on the website, for example "Rs 7,500,000", "85,000 km" and "1,500 cc", so they must be parsed before modelling. Edition and Body Type are missing for about 15.0% and 8.8% of records respectively.

2.2 Features and target variable

Table 1. Predictors and target used in all experiments.

Type	Fields	Meaning
Categorical (7)	Brand, Model, Edition, Transmission, FuelType, BodyType, Negotiable	Manufacturer, model line and trim, drivetrain and listing characteristics (57 brands, 472 models).
Numeric (3)	ManufactureYear, EngineCC, MileageKM	Vehicle age proxy, engine size / segment proxy and accumulated use.
Target (1)	PriceLKR	Seller-advertised asking price in Sri Lankan rupees.

The cleaned dataset is dominated by hatchbacks (35%), SUVs (30%) and saloons (19%); petrol (64%) and hybrid (25%) vehicles; and automatic transmissions (82%). Half of the listings are for vehicles manufactured in 2023 or later, which reflects the re-opening of vehicle imports in 2025.

2.3 Ethical data use

- Only **publicly visible, non-personal vehicle attributes** were collected. Seller names, phone numbers, locations, free-text descriptions and photographs were not scraped, so the dataset contains no personal or sensitive data.
- The data are used for academic, non-commercial research only. The raw file is supplied with the submission so the work can be reproduced, but it is not republished publicly.
- Scraping was limited to the information needed for the task; ikman.lk terms of use should be respected before any further collection or redistribution.

2.4 Preprocessing in Notebook 01

Cleaning was performed before any modelling and is recorded in cleaning_report.json so that every rule and its effect can be audited.

Table 2. Standardisation and parsing.

Technique	What was implemented	Why
Column-name normalisation	Renamed fields, e.g. Year of Manufacture → ManufactureYear, Engine Capacity (cc) → EngineCC, Price Rs. → PriceLKR.	Stable variable names across notebooks, saved files and the app.
Categorical normalisation	Trimmed and collapsed spaces, title case, placeholders mapped to "Unknown".	Avoid duplicate categories for text variants without inventing labels.
Numeric parsing	Removed currency symbols, units and thousands separators with a regular expression; converted with pd.to_numeric(errors="coerce").	Turn website text such as "Rs 7,500,000" into numbers.
Traceability	Kept a normalised copy before exclusions.	Before/after comparisons and a reproducible audit.

Table 3. Data-quality rules and their observed effect.

Quality rule	Treatment	Effect
Identical normalised rows	Keep first occurrence	1,833 rows removed
Missing or non-positive price	Exclude (invalid target)	0 rows
Price above LKR 1 billion	Exclude as a data-entry error (a 2023 Mazda 3 at LKR 21.5 billion)	1 row removed
Year outside 1950 – current year	Exclude	0 rows
Zero mileage on vehicles built up to 2022	Treat as placeholder; set to missing	8 cells

Quality rule	Treatment	Effect
Mileage above 800,000 km	Set to missing (row kept)	13 cells
Engine capacity of electric vehicles	Not applicable; set to missing	85 cells
Engine capacity outside 100–8,000 cc	Set to missing	9 cells
Price below LKR 2M for a 2023+ vehicle	Flag for review; kept	3 records flagged

After cleaning, 94 EngineCC and 21 MileageKM values are missing. They are deliberately **not** imputed in Notebook 01: median imputation and encoding are fitted inside the modelling pipeline on training data only (Section 4.2), so validation and test data never influence learned preprocessing statistics. The cleaned file has **8,554 records** (82.35% of the raw data) and is exported as cleaned_vehicle_data.csv. No percentile-based price trimming was applied because expensive luxury listings can be genuine.

2.5 Exploratory data analysis

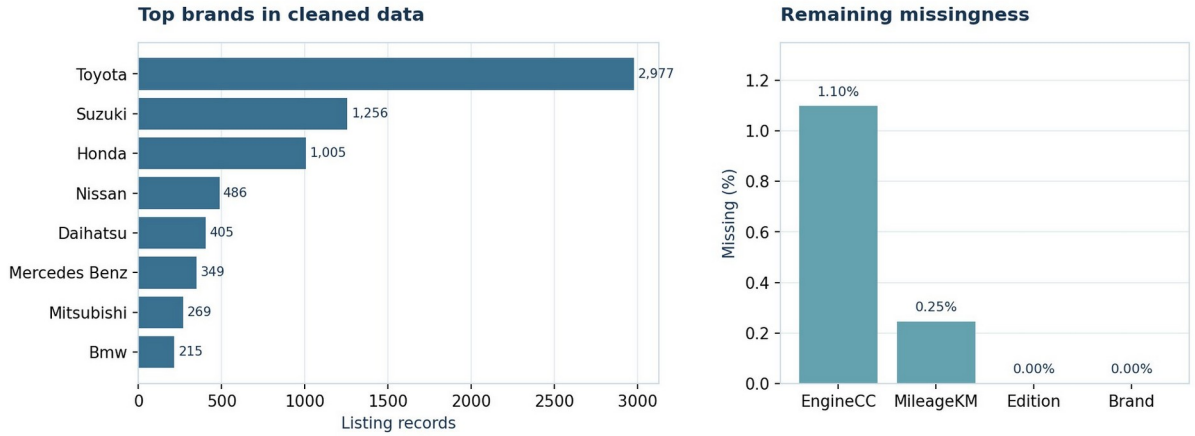


Figure 2. Most represented brands and remaining numeric missingness after cleaning (Notebook 01).

Toyota (2,977 listings), Suzuki (1,256) and Honda (1,005) dominate the data, so there is much less evidence for uncommon brands.

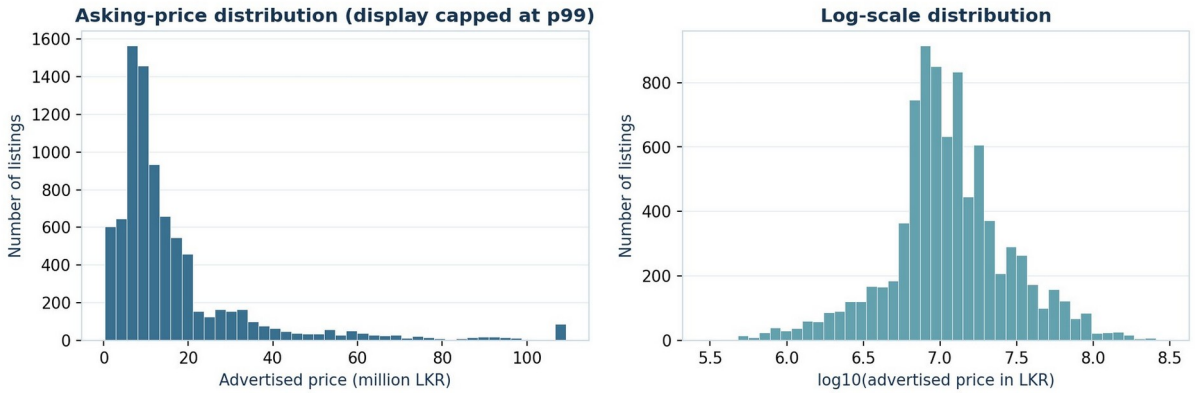


Figure 3. Asking-price distribution (display capped at the 99th percentile) and log-scale view. The cap is for display only.

Table 4. Descriptive statistics of the cleaned target.

Statistic	Min	Q1	Median	Mean	Q3	P99	Max
PriceLKR (million)	0.30	7.15	10.69	17.40	18.44	109.2	300.0

The mean is far above the median, showing a strongly right-skewed target with a long luxury tail. This motivated a log-target for the tree models and the use of RMSE alongside MAE, because RMSE highlights large individual errors.



Figure 4. *Asking price against manufacture year and mileage (sampled listings, display limited to LKR 130M).*

Manufacture year is positively associated with price (Spearman ρ ≈ 0.55) and mileage negatively (ρ ≈ −0.49), but with wide spread: brand, model and configuration change the price of otherwise similar vehicles. These are descriptive associations, not causal effects, and they justify multivariate, nonlinear models.

3. Selection of a New Machine Learning Algorithm

3.1 The Explainable Boosting Machine

The new algorithm is the **Explainable Boosting Machine (EBM)** from Microsoft's open-source InterpretML library (interpret-core 0.7.8). EBM is a modern form of a **generalised additive model (GAM)**. It predicts the target as a baseline plus a separate learned function of each feature:

$$\hat{y} = \beta_0 + f_1(Brand) + f_2(Model) + \ldots + f_9(EngineCC) + f_{10}(MileageKM)$$

Each shape function f_j is learned with **cyclic gradient boosting**: in every round the algorithm visits the features one at a time and fits a very shallow tree on that single feature to the current residuals, using a small learning rate. Because only one feature is used at a time, the order of features does not matter and the effects of correlated features are shared fairly. After thousands of such small steps, the trees for each feature are collapsed into one lookup table (a step function for numeric features, a value per category for categorical ones). Bagging over several outer bags and early stopping on internal validation splits reduce overfitting and provide an uncertainty band for each curve.

EBM is not a deep learning model; it is a tree-based additive model that trains in seconds on this dataset.

3.2 Why EBM was selected

- **Interpretable by construction.** With interactions disabled (interactions = 0), every prediction is an exact sum of a baseline and ten feature contributions. No post-hoc approximation such as SHAP or LIME is needed, so the explanation shown to a user is the model itself.
- **Suited to the data.** It handles mixed categorical and numeric features natively, models nonlinear effects (for example, the steep price increase for recent vehicles), and keeps missing values as their own bin instead of requiring imputation.
- **Research question.** Tree ensembles are usually the most accurate tabular learners but are black boxes. Including EBM tests how much accuracy, if any, a fully transparent model gives up on this problem, and whether both kinds of model learn the same price drivers.
- **Not taught in the module** and compatible with the scikit-learn API, so it fits into the same training, tuning and evaluation pipeline as the reference models.

3.3 How EBM differs from standard models

Table 5. EBM compared with standard models.

Model	Structure	Interpretability	Key difference from EBM
Linear / logistic regression	One weight per (encoded) feature	High, but effects are straight lines	EBM learns a flexible, nonlinear curve per feature instead of one slope.
Decision tree	One tree splitting on many features	Readable only when shallow	EBM never mixes features inside a tree, so each effect can be read separately; a deep tree (depth 18 here) is not readable.
k-NN	Averages prices of similar listings	Low; no global model	EBM learns an explicit global model and is insensitive to feature scaling.
Random Forest / XGBoost / LightGBM	Hundreds of trees, each using many features	Black box; needs post-hoc XAI	EBM boosts one feature at a time and collapses the trees into exact, additive shape functions.

4. Model Training and Evaluation

4.1 Train / validation / test split

Many advertisements share identical specifications. A plain random split could put the same vehicle configuration in both training and test data and overstate performance. Each listing was therefore given a group ID by hashing all ten predictor values, and two GroupShuffleSplit operations produced group-disjoint partitions of roughly 70 / 15 / 15%. Identical feature combinations always fall in the same partition. Group IDs use the predictors only, never the price.

Table 6. Group-disjoint data partitions (random seed 42).

Partition	Listings	Role
Training	5,995	Fit models and all data-dependent preprocessing
Validation	1,274	Compare candidates and select the deployed model
Test	1,285	Single, final estimate of performance after selection

Hyperparameter tuning uses three-fold GroupKFold inside the training partition. For the final test, every candidate is refitted on training plus validation data (7,269 listings).

4.2 Input representations

All six candidates receive the same ten raw features and the LKR target and return predictions in LKR, so a single training, selection and evaluation loop serves every model. Internally, two representations are used, each wrapped in a scikit-learn TransformedTargetRegressor:

Table 7. Input representations.

Component	Black-box models (tree, forest, boosting)	Glass-box EBM
Categorical features	Most-frequent imputation; OneHotEncoder(min_frequency=15, handle_unknown="infrequent_if_exist")	Native nominal terms, no encoding
Numeric features	Median imputation with missing indicators; RobustScaler	Native binning (max 128 bins); missing kept as a separate bin
Target	log(1 + price) for fitting, back-transformed with exp(·) − 1	Price in million LKR, identity link

Component	Black-box models (tree, forest, boosting)	Glass-box EBM
Leakage control	All steps inside one Pipeline, refitted on each training fold	Binning learned from training data only

EBM's target scale was chosen for interpretability: with an identity link, its contributions can be read directly as rupees. This also means the two families optimise different objectives, which is relevant when comparing their accuracy.

4.3 Hyperparameter choices

Table 8. Initial model configurations (fixed in advance, seed 42).

Model	Settings	Rationale
Median baseline	Predicts the training median	Minimum benchmark
Decision Tree	max_depth 18, min_samples_leaf 4, min_samples_split 10	Limit complexity of a single tree
Random Forest	160 trees, max_depth 24, max_features 0.85, min_samples_leaf 2	Variance reduction through bagging
XGBoost	450 trees, depth 7, learning rate 0.055, subsample 0.85, colsample 0.85, L2 = 4, min_child_weight 3	Regularised sequential boosting
LightGBM	380 trees, 31 leaves, depth 10, learning rate 0.05, min_child_samples 25	Efficient leaf-wise boosting
EBM	interactions 0, outer_bags 4, learning rate 0.04, max_rounds 1,200, early stopping 60 rounds, min_samples_leaf 8, max_bins 128	Pure additive model; early stopping chooses the number of rounds

The two strongest ensembles on validation (XGBoost and Random Forest) were then tuned with RandomizedSearchCV (three sampled settings each, three-fold GroupKFold, negative MAE), with preprocessing inside the cross-validated pipeline. EBM kept its fixed configuration because early stopping already selects its number of boosting rounds. The search was deliberately compact and is not exhaustive.

Table 9. Group-aware cross-validation results.

Tuned model	Mean CV MAE (LKR)	CV SD (LKR)	Best sampled settings
Random Forest	1,989,873	93,479	120 trees, max_depth 24, min_samples_leaf 1
XGBoost	2,037,420	47,459	450 trees, max_depth 8, learning rate 0.035, min_child_weight 4

4.4 Performance metrics

Table 10. Regression metrics.

Metric	Definition	Why it is reported
MAE	Mean |actual − predicted| (LKR)	Average error in rupees; primary selection metric
RMSE	√mean (actual − predicted)² (LKR)	Penalises large individual misses more strongly
R^2	1 − SS_residual / SS_total	Share of price variance explained, relative to predicting the mean
Median APE	Median of 100 × |actual − predicted| / actual	Typical percentage error, robust to the luxury tail

4.5 Validation results and model selection

Table 11. Validation performance (1,274 listings), sorted by MAE.

Model	MAE (LKR)	RMSE (LKR)	R^2	Median APE
XGBoost (selected)	**1,871,809**	**5,453,882**	**0.927**	**5.92%**
XGBoost · tuned	1,903,033	5,564,354	0.924	5.88%
Random Forest · tuned	1,942,901	7,303,184	0.869	5.48%
Random Forest	2,023,708	7,358,898	0.867	5.61%
LightGBM	2,202,386	6,772,247	0.887	6.73%
Decision Tree	2,588,351	10,110,665	0.748	6.60%
EBM	4,881,939	9,720,740	0.767	24.37%
Median baseline	10,694,016	21,264,576	−0.114	44.21%

Every configured candidate, EBM included, and every tuned variant competed on validation MAE. The **default-settings XGBoost** had the lowest validation MAE and became the deployed price model; the compact tuning did not improve on it. EBM is carried forward alongside XGBoost regardless of its rank, because it supplies the exact explanations used in Section 5 and in the app.

4.6 Held-out test results

Table 12. Held-out test performance (1,285 listings; models refitted on train + validation).

Model	MAE (LKR)	RMSE (LKR)	R^2	Median APE
Random Forest	1,878,725	6,694,041	0.900	5.37%
XGBoost (selected)	**1,970,645**	**6,678,516**	**0.901**	**6.12%**
LightGBM	2,163,140	7,127,195	0.887	6.68%
Decision Tree	2,199,151	7,487,122	0.875	6.30%
EBM (glass box)	4,987,224	10,511,416	0.755	23.91%
Median baseline	10,531,084	22,199,013	−0.095	43.33%

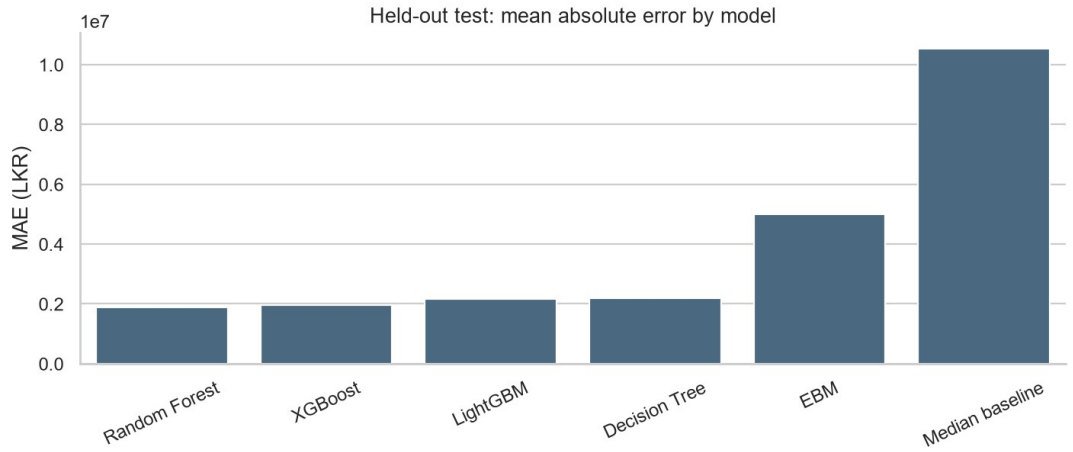


Figure 5. *Held-out test MAE for all candidates (lower is better).*

What the results indicate. All tree models are far better than the median baseline, which confirms that the vehicle attributes carry strong price information. The selected XGBoost explains about 90% of the price variance on unseen listings, with a typical (median) error of about 6% of the asking price. Random Forest has a slightly lower

test MAE (LKR 1.88M against 1.97M), but its RMSE is slightly higher; the difference is small, and the selection is not changed after seeing test results, which keeps the test estimate unbiased.

EBM is clearly less accurate: its test MAE is about 2.5 times that of XGBoost, and its validation MAE (LKR 4.88M) is very similar, so this is not a test-set artefact. Two causes stand out. First, EBM adds feature effects on the raw price scale, whereas the log-target ensembles effectively multiply them; for cheap vehicles, the large deductions for an old year or a small engine can outweigh the baseline, and 103 of 1,285 EBM test predictions (8%) were at or below zero and clipped to zero. Second, with interactions disabled EBM cannot learn that, for example, the effect of age differs between a Toyota and a Mercedes-Benz.

4.7 Error analysis by price band

Table 13. Test MAE by actual price band.

Asking-price band	Test listings	XGBoost MAE (LKR)	EBM MAE (LKR)
Below LKR 10M	601	591,597	3,597,847
LKR 10M – 30M	514	1,383,397	3,919,748
LKR 30M – 100M	156	7,146,686	9,340,922
LKR 100M and above	14	25,055,715	55,310,183

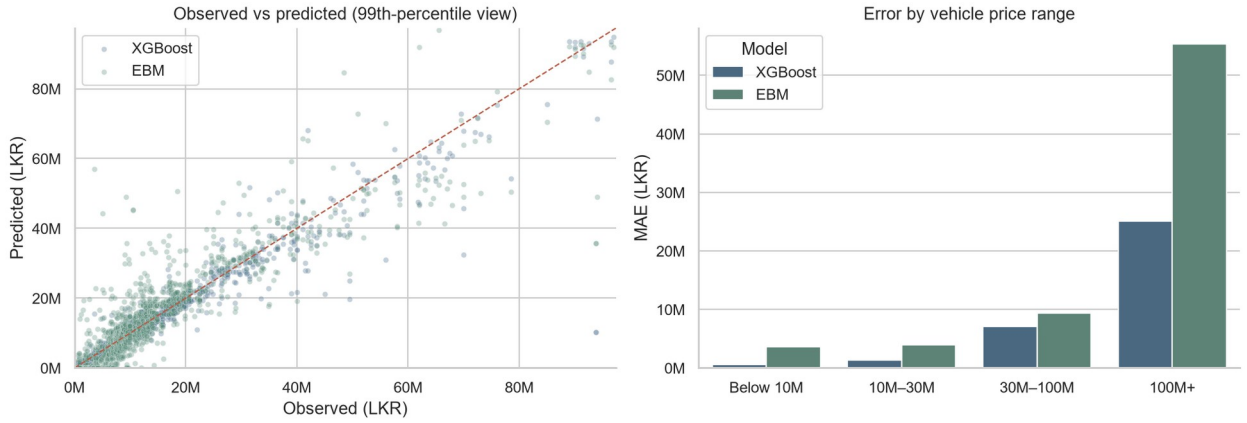


Figure 6. Observed versus predicted prices (left) and MAE by price band (right) for XGBoost and EBM on the test set.

Absolute error grows sharply with price. For the 14 test vehicles above LKR 100M, the XGBoost MAE is about LKR 25 million. A good global R^2 can therefore hide large financial misses for rare luxury cars, and the gap between RMSE and MAE confirms that a small number of very large residuals exist. EBM has a higher error than XGBoost in every band.

5. Explainability and Interpretation

The selected XGBoost model is a black box: each prediction passes through one-hot encoding, hundreds of trees and a log back-transform, so it can only be explained after the fact. EBM is explainable by construction. The analysis pairs the two models around three questions.

Table 14. XAI methods applied.

Question	XGBoost (black box)	EBM (glass box)
Which features matter overall?	Permutation importance	Permutation importance and built-in term importance
How does price respond to a feature?	–	Learned shape functions
Why this price for this vehicle?	–	Exact additive local explanation

5.1 Global feature importance

Permutation importance shuffles one feature at a time in a fixed sample of 450 validation listings (two repeats) and records how much MAE increases. The same procedure is applied to XGBoost and EBM, so their rankings are directly comparable. EBM's **built-in term importance** (mean absolute contribution, in million LKR) is a different quantity and is reported separately.

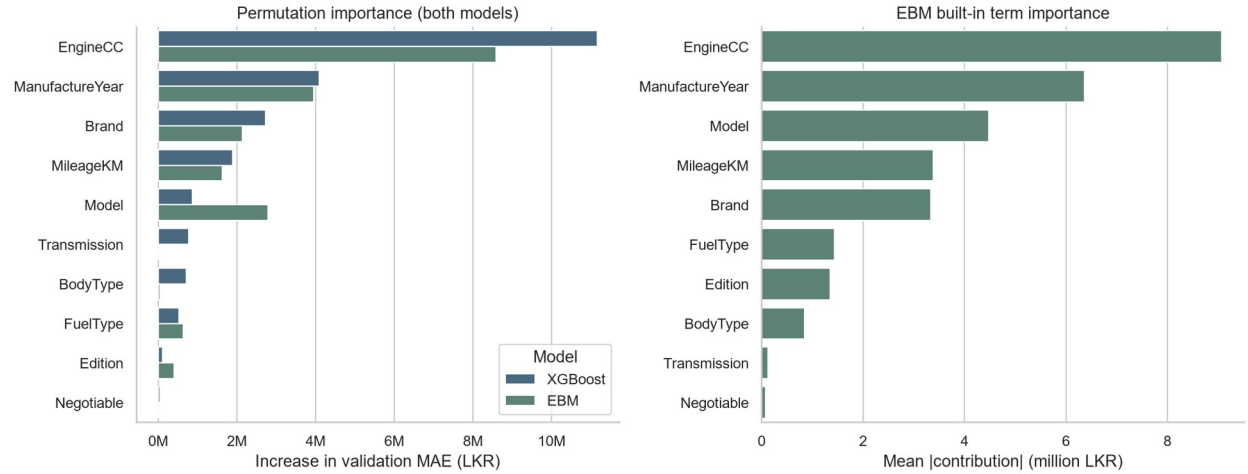


Figure 7. Permutation importance for both models (left) and EBM built-in term importance (right).

Table 15. Global importance values.

Feature	XGBoost: MAE increase (LKR M)	EBM: MAE increase (LKR M)	EBM mean |contribution| (LKR M)
EngineCC	11.15	8.58	9.07
ManufactureYear	4.10	3.95	6.37
Brand	2.73	2.15	3.33
MileageKM	1.89	1.63	3.38
Model	0.87	2.78	4.48
Transmission	0.78	0.00	0.13
BodyType	0.71	0.04	0.85
FuelType	0.53	0.63	1.44
Edition	0.11	0.41	1.35
Negotiable	0.04	0.01	0.08

Both models rank the **same five features** highest: engine capacity, manufacture year, brand, mileage and model. They differ on transmission and body type, which matter to XGBoost but hardly at all to EBM, probably because EBM's per-model term already captures them (a given model usually has one body type and transmission). Because the two models broadly agree on the main drivers, EBM's transparent account is also informative about what the black-box model relies on.

5.2 What the model learned: EBM shape functions

Each EBM shape function shows how much a feature adds to or subtracts from the baseline at every value, in million LKR. Because the model is additive, these curves are the model itself, not an approximation. The shaded bands show the variation across EBM's bagged models.

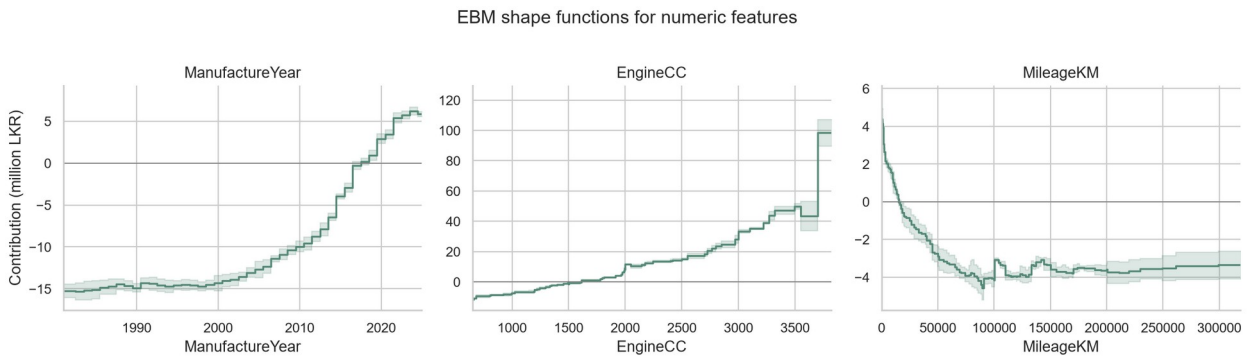


Figure 8. EBM shape functions for manufacture year, engine capacity and mileage (central 98% of training values).

- **Manufacture year:** almost flat until about 2000, then rising steeply and crossing zero around 2015. A 2024–2025 vehicle adds about LKR 5–6M to the baseline, while a pre-2000 vehicle subtracts about LKR 15M.
- **Engine capacity:** rises steadily with displacement, from about −LKR 10M for small engines to roughly +LKR 35–45M at 3,000–3,500 cc. Large engines mark luxury SUVs and premium saloons.
- **Mileage:** falls sharply over the first ~80,000 km (from about +LKR 4.5M at zero to about −LKR 4M) and then levels off. The first owner and first years of use cause the largest loss of value.

5.3 Why this price for this vehicle: local explanation

For a single listing, EBM's intercept plus its per-feature contributions reconstructs its prediction exactly (checked with an assertion in Notebook 02). The example is the held-out listing whose asking price is closest to the test median, chosen without reference to any model's error: a **2025 Toyota Roomy** advertised at LKR 10.98M. XGBoost predicted LKR 11.59M and EBM LKR 13.49M.

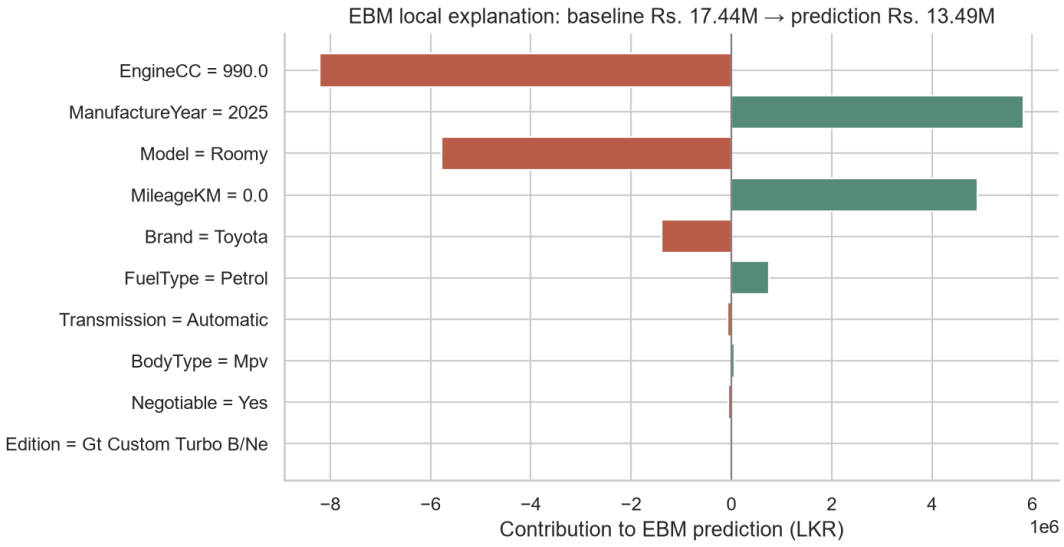


Figure 9. EBM local explanation for a 2025 Toyota Roomy: baseline LKR 17.44M → prediction LKR 13.49M.

Table 16. Exact additive decomposition of the EBM estimate.

Feature = value	Contribution (LKR)
Baseline (intercept)	17,441,261
EngineCC = 990 cc	−8,214,569
ManufactureYear = 2025	+5,833,280
Model = Roomy	−5,781,030
MileageKM = 0	+4,910,504
Brand = Toyota	−1,390,787

Feature = value	Contribution (LKR)
FuelType = Petrol	+755,418
Other four features	−67,889
EBM prediction	**13,486,189**

The small 990 cc engine and the budget Roomy model pull the estimate well below the average listing, while the brand-new year and zero mileage push it back up. The same breakdown is generated live in the Streamlit app for any vehicle the user enters. These contributions explain EBM's estimate only; they do not explain the XGBoost price.

5.4 Does the behaviour match domain knowledge?

- **Yes for age and mileage.** Newer, low-mileage cars are advertised at higher prices, and the steep rise after about 2015 is consistent with the import restrictions that made recent vehicles scarce in Sri Lanka.
- **Yes for engine capacity,** with a caveat. Larger engines mark higher vehicle segments, and Sri Lankan import duty rises with engine capacity, so a strong positive effect is expected. However, EngineCC also acts as a proxy for segment and luxury, so its importance is partly borrowed from information about the type of car.
- **Brand and model** carry market reputation and segment information, as expected; the negative Toyota contribution in the example is likely because most Toyota listings in this dataset are affordable compact models.
- **Negotiable** has almost no effect, which is plausible: it describes the selling style, not the vehicle.

All importance values and contributions describe what the models learned from advertised prices. They are associations, not causal effects: changing a car's engine would not change its price by the amount shown.

6. Critical Discussion

6.1 Limitations of the model

- **Accuracy versus transparency.** The most accurate model (XGBoost) is a black box, and the transparent model (EBM) is about 2.5 times less accurate. Users must choose between the more accurate estimate and the fully explained one; the app shows both side by side.
- **Large errors for expensive vehicles.** MAE is about LKR 25M above LKR 100M, and only 14 test listings fall in that band, so performance there is uncertain.
- **Point estimates only.** No validated prediction intervals are provided.
- **Compact tuning and one split.** Scores depend on the particular split and a small search budget; nested group-aware cross-validation would give more robust estimates.
- **EBM design.** A raw-price target and no interaction terms limit EBM; a log-price EBM or one with a few pairwise interactions is a natural next step.

6.2 Data quality issues

- Prices are **seller-entered asking prices**, not completed sales. Some may be stale, mistyped or leasing down-payment amounts.
- The scrape has **no listing date or advertisement ID**, so reposted ads cannot be fully detected and time-based validation is impossible. Group-aware splitting removes identical feature rows but not slightly edited reposts.
- Mileage, edition and body type are self-reported and sometimes missing or implausible; cleaning thresholds are documented study choices, not verified market rules.

- There are **no condition, location or accident-history fields**, and no time-varying factors such as exchange rates or import tax changes.

6.3 Risks of bias and unfairness

- **Brand imbalance:** Toyota, Suzuki and Honda make up 61% of listings, so predictions for rare brands, editions and luxury cars are less reliable.
- **Snapshot bias:** half the listings are 2023 or newer vehicles from a specific market period; the model may not transfer to a different market phase.
- **Feedback risk:** if marketplaces used the model to suggest prices, today's asking prices could become self-reinforcing.
- No personal attributes of sellers or buyers are used, so the model cannot discriminate against individuals directly.

6.4 Real-world impact and ethical considerations

A transparent price reference can make the used-car market fairer by helping inexperienced buyers spot inflated prices and by helping sellers set realistic ones. The explanations let users challenge an estimate rather than accept it blindly. However, the tool must not be used as a professional valuation for loans, insurance claims, taxation or legal disputes. The app labels itself as an academic prototype and states that estimates are indicative asking prices. Any further data collection should follow ikman.lk terms of use, collect only non-personal vehicle attributes and record listing dates and IDs so that the model can be monitored and retrained over time.

7. Front-End Integration (Streamlit Web App)

The trained models are integrated into a Streamlit web application (app.py). It loads the two saved models (trained_vehicle_price_model.pkl for XGBoost and ebm_vehicle_price_model.pkl for EBM), the cleaned data for dropdown options and the saved evaluation results. Because each saved model is a complete pipeline, the app applies exactly the same preprocessing used in training.

Table 17. Streamlit app structure.

Tab	What the user can do
Estimate price	Choose brand, model (filtered by brand), edition, year, mileage, engine capacity, fuel, transmission, body type and negotiable flag; click Predict to get the XGBoost estimate and the EBM estimate in LKR.
Explainable AI	See the EBM local explanation for the entered vehicle (baseline plus signed contribution of each feature), permutation importance for both models, EBM term importance and shape functions.
Model comparison	Validation and test metrics for all six candidates and the selection rule.
Method and limitations	Data source, cleaning, validation design and responsible-use warnings.

Live deployment. The application is live and hosted on **Streamlit Community Cloud**, so it can be used in any web browser without installing anything:

https://srilankavehiclepricepredictionproject-qpxf6lfrt2547rgriw7pir.streamlit.app/

Streamlit Community Cloud installs the packages in requirements.txt and serves app.py with the saved models in vehicle_ml_outputs/. The app can also be run locally with python -m streamlit run app.py. The accompanying 3–5 minute demo video shows a live prediction and both kinds of explanation.

8. Reproducibility

1. Install dependencies with python -m pip install -r requirements.txt (Python 3.11–3.13; tested with scikit-learn 1.8, XGBoost 3.4, LightGBM 4.7 and interpret-core 0.7.8).

2. Run **Notebook 01** (01_Vehicle_Data_Cleaning_EDA_Submission.ipynb) on ikman-scrapes.csv to create cleaned_vehicle_data.csv and cleaning_report.json.
3. Run **Notebook 02** (02_Vehicle_Price_Regression_Submission.ipynb) to split, train, tune, select, test and explain all six models and save the models, metrics and figures to vehicle_ml_outputs/.
4. Launch the app with python -m streamlit run app.py.

The complete source code (both notebooks, app.py, requirements.txt, the dataset and the saved outputs) is available on GitHub:

https://github.com/VDARW-UOM/Sri_Lanka_Vehicle_Price_Prediction_Project

All random operations use seed 42. Schema, split-size and explanation-reconstruction checks are built into Notebook 02 as assertions. Different library versions may change scores slightly, but the selection rule stays the same.

9. Conclusion

The project delivers a complete, explainable price-estimation workflow for Sri Lankan used cars: scraping-based data collection from ikman.lk, auditable cleaning, leakage-aware group splits, a fair comparison of six regressors, unbiased held-out testing, global and local explanations, and a working web front end. XGBoost is the most accurate price estimator (test MAE LKR 1.97M, R^2 0.901). The new algorithm, the Explainable Boosting Machine, is less accurate (test MAE LKR 4.99M) but gives exact, human-readable explanations, and its explanations agree with XGBoost on the five main price drivers. The key finding is therefore a clear, measured trade-off between accuracy and built-in interpretability, and a practical way to combine both: an accurate model for the estimate and a glass-box model to explain the drivers behind it.

References

- Lou, Y., Caruana, R., Gehrke, J. and Hooker, G. (2013). Accurate intelligible models with pairwise interactions. *Proceedings of the 19th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 623–631.
- Nori, H., Jenkins, S., Koch, P. and Caruana, R. (2019). InterpretML: A unified framework for machine learning interpretability. *arXiv:1909.09223*. https://interpret.ml/
- Hastie, T. and Tibshirani, R. (1990). *Generalized Additive Models*. Chapman & Hall.
- Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32.
- Chen, T. and Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD Conference*, 785–794.
- Ke, G. et al. (2017). LightGBM: A highly efficient gradient boosting decision tree. *Advances in Neural Information Processing Systems 30*.
- Pedregosa, F. et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830. https://scikit-learn.org/
- Project source code repository (GitHub): https://github.com/VDARW-UOM/Sri_Lanka_Vehicle_Price_Prediction_Project
- Live Streamlit application (Streamlit Community Cloud): https://srilankavehiclepricepredictionproject-qpxf6lfrt2547rgriw7pir.streamlit.app/
- ikman.lk. Online classifieds marketplace, Sri Lanka – source of the scraped car advertisements. https://ikman.lk
- Project artefacts: 01_Vehicle_Data_Cleaning_EDA_Submission.ipynb, 02_Vehicle_Price_Regression_Submission.ipynb, app.py and vehicle_ml_outputs/ (cleaning_report.json, validation_metrics.csv, cross_validation_results.csv, test_metrics.csv, error_by_price_band.csv, feature_importance.csv, ebm_global_importance.csv, ebm_example_local_explanation.csv).